

CONTENTS

Transaction processing:

Concurrency control,

ACID property,

Serializability of scheduling,

Locking and timestamp-based schedulers,

Multi-version and optimistic Concurrency Control schemes,

Database recovery.

TRANSACTION MANAGEMENT

- The **transaction** refers to a collection of operations that form a single logical unit of work.
- Transaction is initiated by a **user program** written in a high-level data-manipulation language (typically SQL), or programming language
- Transaction is delimited by statements (or function calls) of the form **begin transaction and end transaction**
- The transaction consists of all operations executed between the **begin transaction and end transaction**.

The database system maintain the following properties of the transactions:

- 1) Atomicity**- Either all operations of the transaction are reflected properly in the database or none are.
 - This property ensures that either all operations of a transaction are executed or it is aborted
 - A transaction can never be completed partially.
 - Each transaction is treated as a single unit (like an atom).

2) **Consistency**- Execution of a transaction in isolation

- with no other transaction executing concurrently preserves the consistency of the database.
- This property of a transaction keeps the database consistent before and after a transaction is completed.
- Execution of any transaction must ensure that after its execution, the database is either in its prior stable state or a new stable state.

3) Isolation:

- A multiple transactions may execute concurrently, the system guarantees that, for every pair of transactions T_i and T_j , it appears to T_i that either T_j finished execution before T_i started or T_j started execution after T_i finished.
- This property states that two transactions must not interfere with each other
- It ensures that the integrity of the database is maintained and don't get **any ambiguous values**.
- This property is enforced by the concurrency control subsystem of DBMS.

4) Durability: After a transaction completes successfully, the changes it has made to the database persist, even if there are system failures.

- This property ensures that the changes made to the database after a transaction is completely executed, are durable
- It indicates that permanent changes are made by the successful execution of a transaction.
- In the event of any system failures or crashes, the consistent state achieved after the completion of a transaction remains intact.

Concurrency Control

- Concurrency control is a very important concept of DBMS
- The simultaneous execution or manipulation of data by several processes or user **without** resulting in **data inconsistency**.
- Concurrency control provides a procedure that is able to control **concurrent execution** of the operations in the database.
- Ensures that concurrent execution of transactions does not result in a **loss of database** consistency.

Concurrency Control Protocols

- Concurrency control protocols are **the set of rules** to solve the concurrency control problems in the database.
- The concurrent execution of a transaction is provided with **atomicity, consistency, isolation, durability**, and serializability via the concurrency control protocols.

1) **Locked based Protocol**

2) **Timestamp based Protocol**

Locked based Protocol

- Each transaction needs to acquire **locks** before they start accessing or modifying the data items.

Two types of locks used in databases:

- 1) Shared Lock
- 2) Exclusive Lock

1) Shared Lock :

- Shared lock is also known as **read lock**
- Allows multiple transactions to **read the data** simultaneously.
- The transaction which is holding a shared lock can only read the data item but it can not modify the data item.

2) Exclusive Lock :

- Exclusive lock is also known as the **write lock**.
- Exclusive lock allows a transaction to **update a data** item.
- one transaction can hold the exclusive lock on a data item at a time.
- While a transaction is holding an exclusive lock on a data item, no other transaction is allowed to acquire a shared/exclusive lock on the same data item.

There are two kind of lock based protocol mostly used in database

Two Phase Locking Protocol :

- Two phase locking is a widely used technique
- strict ordering of lock acquisition and release.

Two phase locking protocol works in two phases

1) Growing Phase :

- In this phase the transaction starts acquiring locks before performing any modification on the data items.
- Once a transaction acquires a lock, that lock can not be released until the transaction reaches the end of the execution

2) Shrinking Phase :

- In this phase, the transaction releases all the acquired locks once it performs all the modifications on the data item.
- Once the transaction starts releasing the locks, it can not acquire any locks further.

Strict Two Phase Locking Protocol

- Strict two phase locking the transactions are only allowed to release the locks only when they performs commits.

Serializability in DBMS

- serializability is a term that is a property of the system that describes how the different process operates the shared data.
- In DBMS, the Serializable schedule is a property in which the read and write operation sequence does not disturb the serializability property.

Types of Serializability

- In DBMS, all the transaction should be arranged in a particular order, even if all the transaction is concurrent.
- If all the transaction is not serializable, then it produces the incorrect result.

1) Conflict Serializability

- Conflict serializability is a type of conflict operation in serializability that operates the same data item that should be executed in a particular order and maintains the consistency of the database.

2) View Serializability

- View serializability is a type of operation in the serializable in which each transaction should produce some result
- These results are the output of proper sequential execution of the data item

Benefits of Serializability in DBMS

1) Predictable execution:

- In serializable, all the threads of the DBMS are executed at one time.
- There are no such surprises in the DBMS.
- In DBMS, all the variables are updated as expected and there is no data loss or corruption.

2) Easier to Reason about & Debug:

- In DBMS all the threads are executed alone, so it is very easier to know about each thread of the database.
- This can make the debugging process very easy

3) Reduced Costs:

- With the help of serializable property, we can reduce the cost of the hardware that is being used for the smooth operation of the database.
- It can also reduce the development cost of the software.

4) Increased Performance:

- In some cases, serializable executions can perform better than their non-serializable counterparts
- allow the developer to optimize their code for performance.

Multiversion concurrency control (MVCC)

- Multi-version protocol aims to reduce the delay for read operations.
- Maintains multiple versions of data items.
- When a write operation is performed, the protocol creates a new version of the transaction data to ensure conflict-free and successful read operations.

The newly created version contains the following information:

- 1) Content** – This field contains the data value of that version.
- 2) Write_timestamp** – This field contains the timestamp of the transaction that created the new version.
- 3) Read_timestamp** – This field contains the timestamp of the transaction that will read the newly created value.

Various Types of MVCC

1) Snapshot-based

- Creates a snapshot of the database at the start of a transaction and uses it to provide necessary data for the transaction.
- Easy to implement
- Significant overhead due to storing multiple versions of data

2) Timestamp-based

- Assigns a unique timestamp to each transaction that creates a new version of a record
- used to determine data visibility to transactions

- More efficient than snapshot-based MVCC
- Requires additional storage to store timestamps

3) **History-based**

- Stores a complete history of all changes made to a record, allowing for easy rollback of transactions
- Provides highest level of data consistency
- Most complex of the MVCC techniques

4) **Hybrid**

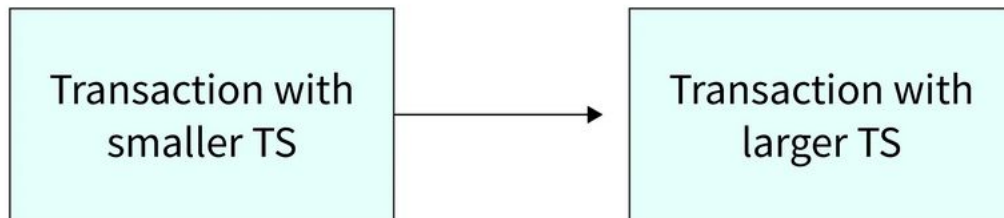
- Combines two or more MVCC techniques to balance performance and data consistency
- Provides benefits of multiple MVCC techniques

Benefits of multiversion concurrency control (MVCC)

- 1) MVCC is a technique that helps databases manage multiple transactions happening at the same time.
- 2) MVCC helps reduce conflicts between transactions accessing the same data.
- 3) The database can allow multiple transactions to read and write data without locking the entire database.
- 4) It improves the speed of reading data.

Timestamp Based Protocols in DBMS

- Timestamp-based protocols in dbms are used to order the transaction in ascending order of their creation time.
- The creation time is the system time or a logical counter.
- For example, if there are two transactions T1 and T2. T1 enters the system at 008 and T2 enters the system at 009 then T1 is given priority over T2.



Timestamp Ordering Protocol working

- **W_TS(X)** (write timestamp) is the largest timestamp of any transaction that executed write(X) successfully.
- **R_TS(X)** (read timestamp) is the largest timestamp of any transaction that executed read(X) successfully.
- If Read timestamp of data item X is greater than the Timestamp of the Transaction T i.e $R_TS(X) > TS(T)$ or
- if Write Timestamp of the data item X is greater than the timestamp of the transaction i.e $W_TS(X) > TS(T)$, then abort and rollback T and reject the operation.

- If Write Timestamp of data item X is greater than the timestamp of the transaction T i.e $W_TS(X) > TS(T)$, then abort and reject T and reject the operation
- If Write Timestamp of data item X is less than or equal to the timestamp of the transaction T i.e $W_TS(X) \leq TS(T)$, then execute the $R_item(X)$ operation of T and set $R_TS(X)$ to the larger of $TS(T)$ and current $R_TS(X)$.

Database Recovery Techniques

- Database recovery techniques are used in database management systems (DBMS) to restore a database to a consistent state after a failure or error has occurred.
- The goal of recovery techniques is to ensure data integrity and consistency and prevent data loss.

Types:

- **Rollback/Undo Recovery Technique**
- **Commit/Redo Recovery Technique**
- **CheckPoint Recovery Technique**

Rollback/Undo Recovery Technique

- The rollback/undo recovery technique is based on the principle of backing out or undoing the effects of a transaction that has not been completed successfully due to a system failure or error.
- undoing the changes made by the transaction using the log records stored in the transaction log.
- The transaction log contains a **record of all the transactions** that have been performed on the database.
- The system uses the log records to undo the changes made by the failed transaction and restore the database to its previous state.

Commit/Redo Recovery Technique

- The commit/redo recovery technique is based on the principle of reapplying the changes made by a transaction that has been completed successfully to the database.
- It is accomplished by using the log records stored in the transaction log to redo the changes made by the transaction that was in progress at the time of the failure or error.
- The system uses the log records to reapply the changes made by the transaction and restore the database to its most recent consistent state.

Checkpoint Recovery Technique

- used to improve data integrity and system stability, especially in databases and distributed systems.
- In the event of a breakdown, the system can be restored to the most recent checkpoint, which reduces data loss and downtime.